Charting the Reasoning Landscape of LLMs via Structured Adversarial Perturbations

Seminar Report

Submitted in partial fulfillment of the requirements

for the degree of

Master of Technology

by

Harshith Matta

(Roll No. 25M0834)

Under the guidance of

Prof. Soumen Chakrabarti

Department of Computer Science and Engineering
Indian Institute of Technology Bombay
May 2026

Acknowledgement

I express my sincere gratitude to my guide Prof. Soumen Chakrabarti, for giving me the opportunity to work on this seminar topic and for his continuous guidance, valuable suggestions, and encouragement throughout the seminar.

I would also like to thank Prof. Arjun Bhagoji for investing his valuable time, sharing insightful suggestions, and supporting me during the course of this work.

I am thankful to the Department of Computer Science and Engineering, IIT Bombay, for providing a supportive academic environment and the necessary resources for this seminar.

Harshith Matta
Computer Science and Engineering
IIT Bombay

Abstract

The recent generation of Large Language Models (LLMs) has excelled at reasoning tasks such as mathematics, logic, and programming. Test-Time Scaling (TTS) has enabled LLMs to achieve this tremendous performance on these tasks. However, there remains significant uncertainty as to whether the high performance of these models reflects superior reasoning abilities, or merely improved proficiency in generating answers consistent with training examples. The standard evaluation paradigm focuses only on the generated answer without considering the trace of reasoning process leading up to the answer. Therefore, current evaluations fail to identify whether the correct answer was produced by valid reasoning. This report surveys ten recent papers covering this evaluation paradox from various perspectives: dynamic problem generation (DYVAL), graph-based trace evaluation (DAG-Math), code-based abstraction analysis (Program-of-Thought), symbolic perturbation experiments on grade school math (GSM-Symbolic), arithmetic stress tests (MATH-401), logic benchmarks (SATBench, ZebraLogic), causal reasoning (causal-graph construction), agentic delegation hazards (document corruption), and a large scale comparison of TTS strategies. In conjunction with the survey, we present an adversarial task, the Pencil-Exchange Task, on Gemini 3.1 Pro. This task requires the model to keep track of how many pencils each person has over time on a varying surface. Our findings concur with those from the survey that, once the problem becomes only slightly more complicated, there is an irregularly decreasing level of accuracy, which seems more like pattern recognition rather than accurate record-keeping. The field, in our view, must have a method of assessment which examines the reasoning trace, creates new scenarios, and explores various domains.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Over the last three years, Large Language Models (LLMs) have made major improvements on reasoning tasks. Models such as OpenAI o1, DeepSeek-R1, Claude 3.7 Sonnet, and Gemini 3.1 Pro now achieve state-of-the-art performance on math reasoning benchmarks (AIME, MATH), competitive programming, and graduate-level question answering (GPQA). Three ideas are behind most of this progress: Chain-of-Thought (CoT) prompting, reasoning-oriented post-training, and Test-Time Scaling (TTS).

Chain-of-Thought prompting asks the model to write intermediate reasoning steps before producing a final answer, either through worked examples (few-shot CoT) or simple instructions such as "let us think step by step" (zero-shot CoT). The model no longer has to compress the entire reasoning process into a single prediction; it can decompose the problem into smaller, more manageable steps.

Reasoning-oriented post-training uses supervised fine-tuning on curated reasoning traces and reinforcement learning to reward correct reasoning paths. Specialised RL algorithms such as Proximal Policy Optimization (PPO) [1] and Group Relative Policy Optimization (GRPO) [2] compare multiple generated solutions and encourage trajectories that perform better relative to other candidates. Models such as DeepSeek-R1 [3] use GRPO-style training to encourage long, self-corrective reasoning chains.

Test-Time Scaling (TTS) lets the model use additional compute at inference — longer reasoning chains, multiple candidate solutions, parallel rollouts, or learned verifiers that pick the best answer among several candidates.

Even with these gains, one major question remains open: do higher benchmark scores mean the model is genuinely reasoning better, or has it simply gotten better at matching patterns from its training data? Two issues make this hard to answer. First, *data contamination*: most public benchmarks live on the open internet, and modern LLMs are trained on internet corpora, so the model may have seen the test items during pretraining. Second, *static complexity*: a fixed benchmark has a fixed difficulty ceiling, and once frontier models saturate it, the benchmark loses its ability to distinguish strong reasoners from weak ones. Both issues are made worse by the standard practice of evaluating only the final answer: a wrong chain can still produce the right answer if errors cancel out, and a correct chain can still slip on the last step. Final-answer accuracy mixes these very different behaviours into a single number.

Recent work attacks these limitations from several directions. *Dynamic evaluation* generates fresh problems at evaluation time, reducing contamination and letting difficulty scale with model capability [4]. *Process-level evaluation* grades the reasoning trace itself rather than only the final answer [5]. *Robustness evaluation* applies controlled perturbations — name swaps, number swaps, paraphrases, distractors — to test whether reasoning is genuine or only pattern matching [6]. Domain-specific work pushes beyond mathematics into symbolic logic [7, 8], causal reasoning [9], agentic document understanding [10],

and arithmetic reasoning [11]. *Abstraction-level* studies use Program-of-Thought prompting to ask whether models that emit code reason at a higher level [12]. *TTS-strategy* studies analyse which inference-time scaling recipes give the best performance under fixed compute budgets [13].

This report has two goals. The first is to bring these papers together into one picture of where LLM reasoning is strong, where it is fragile, and where current evaluation methods fall short. The second is to present our own experiment — the *Pencil-Exchange* task — which applies the lessons from the surveyed literature to a controllable state-tracking problem on Gemini 3.1 Pro.

The rest of the report is organised as follows. Chapter 2 covers CoT prompting, TTS, and a summary of the surveyed works. Chapters 3 to 7 review the surveyed papers in detail. Chapter 8 presents the Pencil-Exchange experiment. Chapter 9 concludes and lists open research directions.

Chapter 2

Background

2.1 Chain-of-Thought Prompting

Chain-of-Thought (CoT) is the simplest of the inference-time reasoning techniques. Instead of asking for the answer in one go, the prompt either shows a few solved examples where each step is written out (few-shot CoT) or just appends a phrase like "let us think step by step" to push the model into writing a step-by-step trace before its final answer (zero-shot CoT). The cost is in extra tokens; the gain is in accuracy on tasks that need composition, search, or multi-step arithmetic.

But CoT only *exposes* the chain of reasoning – it does not check that the chain is correct. A CoT trace can be smooth, fluent, and self-consistent, and still wrong. So CoT is best thought of as a vehicle for reasoning, not a guarantee that the reasoning is right. The main concern of this report is to evaluate what is actually being carried in that vehicle.

2.2 Test-Time Scaling (TTS)

Test-Time Scaling is the umbrella term for techniques that spend extra compute at inference instead of at training time. TTS methods fall into three families.

2.2.1 Parallel Scaling

The model is sampled N times independently, giving N candidate answers (and N candidate chains). The candidates are then combined into a single answer. The simplest combiner is *majority voting* (also called *self-consistency*), which returns the most common answer. A stronger combiner is *best-of-N*, which uses a learned verifier (a reward model or a process reward model) to score each candidate and return the best. Beam search, Frontier-First-Search (FFS), and Lowest-First-Search (LFS) refine the search frontier instead of sampling independently, but they all share the parallel character.

2.2.2 Sequential Scaling

A single, very long trace is generated, and the model is encouraged to backtrack, revise earlier steps, or check itself. "Thinking" models like GPT-o1 and DeepSeek-R1 are trained to produce these long traces by default. Tree-structured search methods (like Monte-Carlo Tree Search, MCTS) interleave generation and pruning to explore the solution space depth-first while still producing one coherent trace.

2.2.3 Hybrid Scaling

A mix of the two: generate N long sequential traces in parallel, then aggregate. This pays the cost of both families but, in principle, gets the breadth of parallel scaling and the depth of sequential scaling.

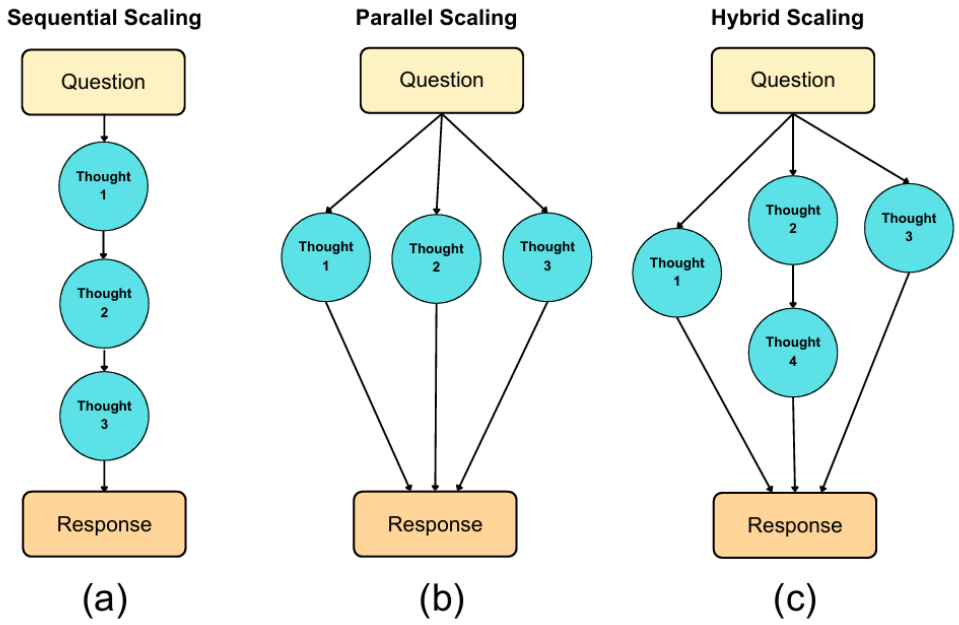

Figure 2.1: The three families of Test-Time Scaling strategies. Parallel scaling samples many independent traces and combines them; sequential scaling produces one long trace that revises itself; hybrid scaling combines the two.

2.3 Static vs. Dynamic Evaluation

Most reasoning benchmarks in use today – GSM8K, MATH, AIME, MMLU – are *static*: a fixed set of problems is released, models are evaluated on it, and scores are reported on a public leaderboard. This setup has two well-known weaknesses. First, items can leak into pre-training corpora, so the benchmark gets contaminated. Second, the difficulty distribution is fixed, so once a model saturates the benchmark the score loses its discriminative power.

Dynamic evaluation, in contrast, generates new problems *procedurally* at evaluation time. Since the problems do not appear in any corpus, contamination is ruled out by construction [4]. Since the generator can be tuned, difficulty can be raised as models get stronger, so the benchmark keeps being informative. The cost is engineering: it is not easy to build a generator whose ground-truth answers are unambiguous and whose generated problems are always solvable.

2.4 Surveyed Works at a Glance

Before going into the details of each paper, Table 2.1 gives a one-line summary of the ten works covered in this report, along with the focus of each and its main finding. The table is meant as a roadmap for the chapters that follow.

Table 2.1: Summary of surveyed works.

Paper	**Focus**	**Key Finding**
DYVAL [4]	Dynamic eval.	Static benchmark scores are inflated by memorisation; model rankings shift on dynamically generated problems.
DAG-Math [5]	Process accuracy	Perfect Reasoning Rate is much lower than pass@1; correct answers often come from flawed chains.
PoT [12]	Abstraction	PoT raises consistency but not correctness; reveals abstraction ceilings in LLMs.
GSM-Symbolic [6]	Robustness	High variance under paraphrase; distractors cause accuracy drops of up to 65%.
MATH-401 [11]	Arithmetic	LLMs are unreliable at multi-digit multiplication, division, and modular arithmetic.
SATBench [7]	Logic	Auto-generated SAT puzzles expose sharp accuracy cliffs as clauses and variables grow.
ZebraLogic [8]	Logic	Performance collapses on 5×5 grids and beyond, even with TTS.
Causal-Graph [9]	Causality	LLMs are biased and inconsistent when asked to build causal graphs from text.
DocCorrupt [10]	Agentic	Delegated document edits silently introduce errors that users rarely catch.
Art-of-TTS [13]	TTS strategy	No single strategy dominates; the best recipe depends on the model class.

Chapter 3

Answer Accuracy is Not Enough: Process-Level Evaluation

The deepest problem with the standard evaluation methodology is that final-answer accuracy is too coarse a signal. A model can reach the right answer for the wrong reason – error cancellation, lucky guessing, surface heuristics. A model can also reach the wrong answer in spite of mostly correct reasoning, just by slipping on the last arithmetic step or by misformatting the output. Process-level evaluation tries to pull these signals apart by looking at the intermediate steps of the reasoning trace.

3.1 DAG-Math: Reasoning Traces as Directed Acyclic Graphs

DAG-Math [5] formalises the idea that a reasoning trace is more than a flat sequence of sentences – it is a *graph* of facts and dependencies. Each intermediate mathematical statement (a definition, a derivation, an arithmetic substitution) becomes a node in a Directed Acyclic Graph (DAG). The edges encode which facts each step depends on. The ground-truth solution to a problem is itself a DAG, derived from a canonical solution. The model's trace is parsed into a candidate DAG, and the two are compared.

The headline metric introduced by DAG-Math is the *Perfect Reasoning Rate* (PRR). PRR is the fraction of problems on which *every* node in the model's reasoning DAG aligns with a node in the ground-truth DAG. PRR is therefore much stricter than pass@k: a model that gets the answer right but takes a wrong intermediate step still scores zero. In practice, the gap between pass@1 and PRR is large – often a factor of two or more across state-of-the-art models. What this means is that a substantial fraction of correct answers are produced by chains that contain at least one factual or logical error. Pass@1 therefore over-estimates reasoning quality, and we need PRR (or a similar trace-aware metric) if we want to compare models on *how* they reason rather than only on *what* they output.

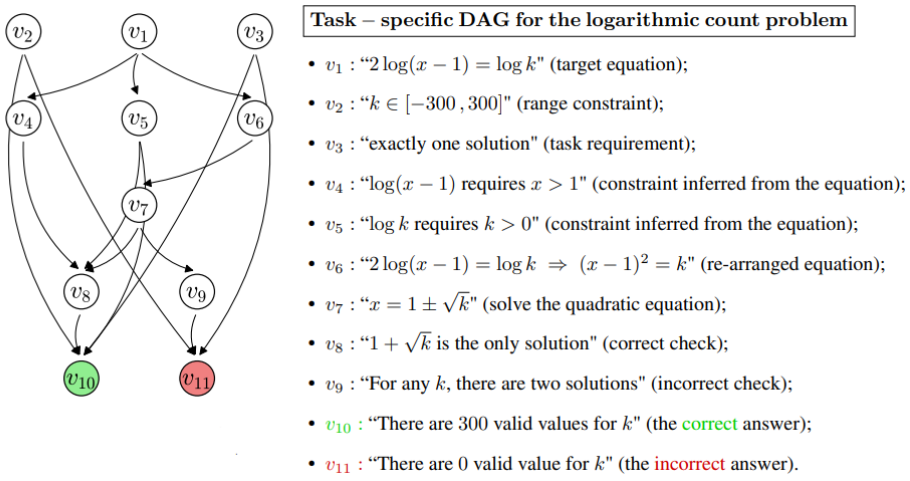

Figure 3.1: DAG-Math represents a reasoning trace as a directed acyclic graph. Nodes are mathematical facts; edges are the dependencies between steps. The Perfect Reasoning Rate (PRR) is the fraction of traces in which every node aligns with the ground-truth DAG.

A second contribution of DAG-Math is methodological. By making the reasoning structure explicit, the framework makes it possible to attribute errors to specific failure modes: missing premises (absent nodes), invalid inferences (wrong edges), and unused hypotheses (orphan nodes). This is much more informative than a single accuracy score and points to specific training-data or fine-tuning fixes for each failure mode.

3.2 Program-of-Thought: Consistency Without Correctness

Program-of-Thought (PoT) [12] replaces natural-language reasoning steps with executable code. Instead of writing "the train travels at 60 km/h for 2 hours, so it covers 120 km," a PoT model emits a Python snippet that computes the same result. The hope is twofold: first, that anchoring reasoning to executable code reduces arithmetic mistakes (the interpreter is exact); second, that the discipline of code forces the model to reason at a higher level of abstraction, treating quantities as named variables instead of as concrete numbers it has seen before.

The authors test this hope by generating many *variants* of each problem – identical in structure but different in surface details – and asking whether a PoT-fine-tuned model gives consistent answers across the variants. They find that PoT does raise *consistency*: a PoT model tends either to get all variants of a problem right or to get all of them wrong. But PoT does *not* raise *correctness*: the rate at which "all variants are correct" is no higher than for plain CoT.

The diagnosis is that PoT models are memorising code templates. A model that produces a correct script for a class of problems will produce a correct script for every variant of that class, which gives the consistency. But the model has not learned to reason its way to a new template when the problem class shifts; it has only learned to retrieve a template that worked before. The authors call this an *abstraction ceiling*: the model can apply the templates it has, but it cannot synthesise new ones. This is a direct, falsifiable form of the broader pattern-matching hypothesis.

Chapter 4

Robustness Under Surface Perturbations

A reasoner that has truly understood a problem's structure should not change its answer when we change things that do not touch that structure. Swapping a person's name from "Alice" to "Anjali," replacing the constant 7 with 13, or paraphrasing "three more than" as "in excess of three" should not move the answer, because none of these edits touches the underlying relationship. If a model's accuracy does move under such edits, the only honest reading is that the model was relying on the surface form of the problem, not its structure.

4.1 GSM-Symbolic: Naming, Numbers, and Distractors

GSM-Symbolic [6] converts problems from the classic GSM8K benchmark into symbolic templates that can be re-instantiated with new names and new numerical values. The same underlying problem can therefore appear in many surface forms, all of which a real reasoner should solve in the same way. The authors report three robustness phenomena.

High variance under paraphrase. When the same problem is re-instantiated dozens of times with different names and slightly reworded prompts, frontier models swing widely on what are nominally identical problems – tens of percentage points between the easiest and hardest paraphrases. This variance is a direct sign that the surface form, not the underlying structure, is driving the model's answer.

Numerical instability. Substituting only the numerical values – without changing names, structure, or wording – is enough to flip many answers. Worse, the leaderboard rank of models reorders relative to GSM8K when this substitution is applied. So a model that ranks highest on GSM8K is not necessarily the best reasoner; it may simply be the best matcher for GSM8K's particular numerical patterns.

Distractor fragility (GSM-NoOp). The most striking finding concerns distractor robustness. The authors create *GSM-NoOp* by adding one or two clauses of irrelevant information to each problem. The clauses are factually correct and on-topic but logically unused: a real reasoner should ignore them. In practice, frontier models fold the distractor into their reasoning chain, which produces accuracy drops of up to 65% on some model and setting combinations. The effect is much larger than naming or number perturbations, which suggests current models have no reliable way of telling relevant from irrelevant content within a single prompt.

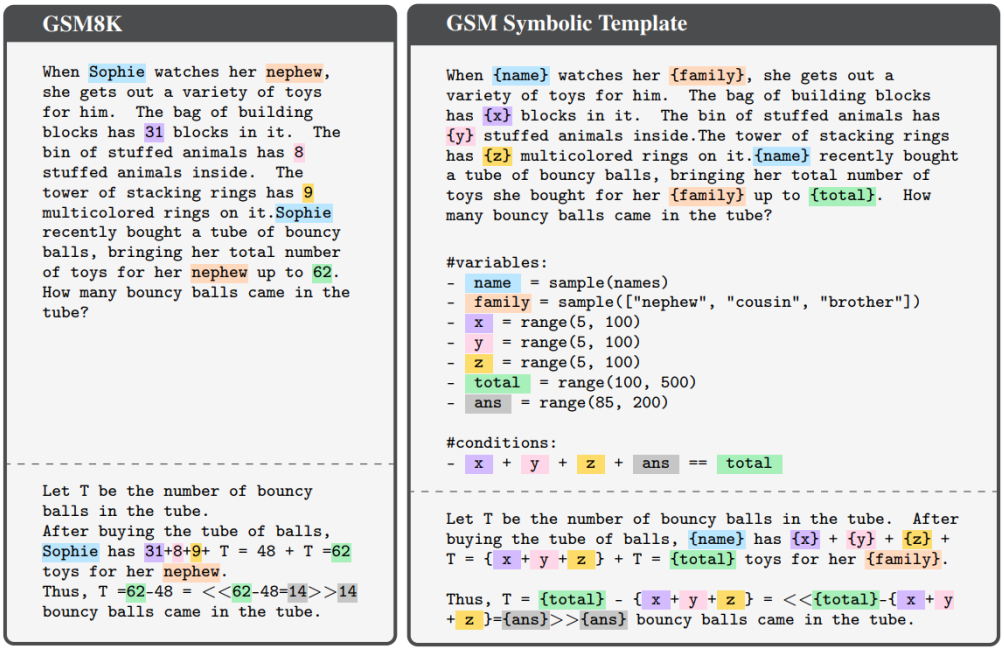

Figure 4.1: Illustration of the GSM-Symbolic template creation process.

GSM-Symbolic shows that GSM8K's static success hides a fragile underlying competence. The models score well on GSM8K because they have, in effect, optimised against GSM8K's particular surface distribution. Once the surface is shaken, the performance turns out to be much weaker.

4.2 MATH-401: Arithmetic at the Level of the Operator

If GSM-Symbolic stresses word-problem reasoning, MATH-401 from Yuan et al. [11] stresses the most basic substrate: arithmetic itself. The benchmark covers the standard operators (addition, subtraction, multiplication, division, exponentiation, modulus, logarithm) over a range of operand sizes. The findings are sobering. LLMs are reliable at small-operand addition and subtraction, but accuracy drops sharply as operands get bigger, especially on multiplication, division, and modular arithmetic. Multi-digit multiplication, in particular, is a weak spot: large frontier models fail on six-digit multiplications that any pocket calculator handles in microseconds.

This matters for higher-level reasoning because most chain-of-thought traces eventually bottom out at an arithmetic step. A model that is unreliable at the arithmetic substrate will be unreliable at the top of the reasoning stack, no matter how good its high-level decomposition is. MATH-401 exposes a quiet but very common failure mode: the chain is fine, the decomposition is fine, but the model miscomputes 437×982 in the last step. The practical implication is that any reasoning evaluation that does not control for arithmetic-substrate errors will mix them up with reasoning errors and confuse both the diagnosis and the fix.

Chapter 5

Dynamic Evaluation: DYVAL

DYVAL [4] is one of the main frameworks for dynamic, contamination-resistant evaluation. The framework rests on three formal components.

Generation function G. The generator produces a Directed Acyclic Graph (DAG) whose nodes encode atomic facts (variables, constants, intermediate values) and whose edges encode the dependencies among them. The DAG *is* the problem: by walking it from leaves to root, one obtains the ground-truth answer. Because the DAG is generated procedurally, no instance has appeared in any pre-training corpus.

Constraint function C. A set of constraints enforces mathematical validity (no division by zero, no negative square roots, etc.) and provides knobs for difficulty control: the depth of the DAG, the branching factor, the number of operations, the operand magnitude. By turning these knobs the evaluator can produce instances ranging from very easy to harder than any problem the model has seen during training.

Description function F. Once the DAG is generated and validated, F renders it into natural language using a templated verbalisation. The output is a problem statement that looks like something a human would write, but it is produced on the fly and is unique to that evaluation run.

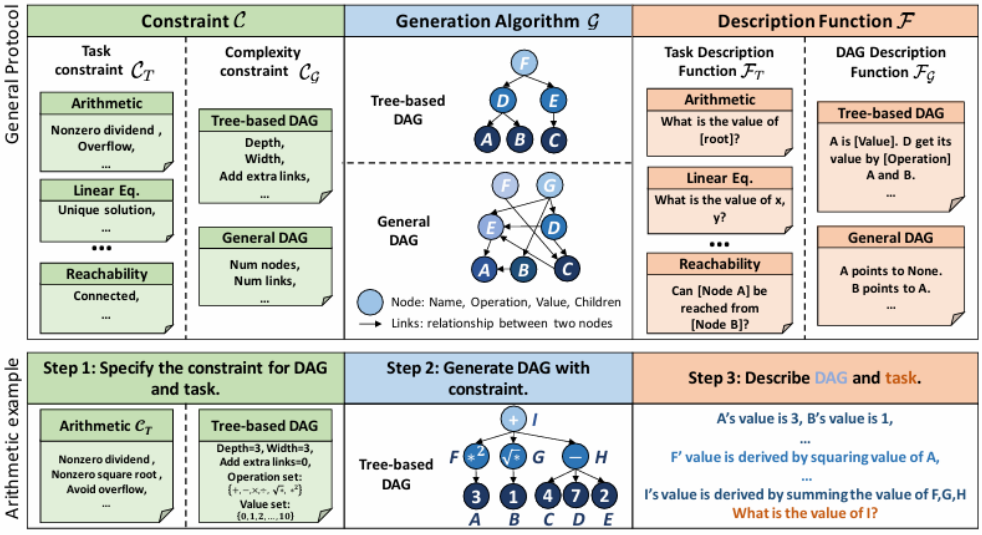

Figure 5.1: DYVAL evaluation pipeline. A generator G produces a DAG, a constraint function C filters it for validity and controls difficulty, and a description function F renders it into natural language. Each evaluation run yields fresh problems unseen by the model.

The empirical results from DYVAL are striking. On dynamically generated problems

calibrated to the same depth as standard benchmarks, frontier models like GPT-4 lose double-digit percentage points compared to their static-benchmark scores. Even more importantly, the relative rankings of models change. A model that ranks higher than another on GSM8K may rank lower on DYVAL-generated arithmetic problems of the same depth. Both observations support the central concern of the paper: static benchmarks *over-estimate* reasoning ability, and the over-estimation is *not the same* across models, so the public leaderboard cannot be trusted to reflect real reasoning quality.

A side benefit of DYVAL's DAG construction is that it provides a natural ground-truth trace. This trace can in turn be used as the reference DAG for trace-level evaluation in the style of DAG-Math. The two frameworks therefore compose: DYVAL produces the problem and the reference DAG, and DAG-Math grades the model's trace against that reference. Together they give what is, in principle, the strongest current evaluation of mathematical reasoning: dynamic, contamination-resistant, difficulty-controlled, and trace-aware.

Chapter 6

Reasoning Beyond Mathematics

Most evaluation work focuses on mathematical reasoning because the answers are unambiguous and the problems are easy to generate. But mathematics is only one kind of reasoning, and a real general-purpose reasoner has to perform across all kinds. This chapter reviews four lines of work that extend evaluation into logic, causality, and agentic document handling.

6.1 SATBench: Logical Reasoning From SAT Formulas

SATBench [7] is a logical reasoning benchmark whose problems come from random Boolean satisfiability (SAT) formulas. The pipeline samples a SAT formula in conjunctive normal form, translates it into a natural-language puzzle ("Either Anita brought the umbrella or Bharat brought the raincoat; if Bharat brought the raincoat then Chitra was wearing boots; ..."), and asks the model to decide whether a satisfying assignment exists and, if it does, to give one. Because the benchmark is built from SAT formulas, problem difficulty can be tuned precisely through the variable count and the clause-to-variable ratio, and the ground truth can be verified by a SAT solver.

The findings of SATBench echo those of DYVAL but in a strictly logical setting. Frontier models do well on small instances (few variables, few clauses), but accuracy drops sharply as the formula grows. Even with chain-of-thought enabled, models often fail on instances that an off-the-shelf solver finishes in milliseconds, which suggests the model is not really running a search procedure – it is just pattern-matching against patterns of SAT-style language. SATBench also exposes an asymmetry: models are more reliable at confirming satisfiability when given a hint about the assignment than at finding the assignment from scratch, which suggests the verifier-style task is well within reach but the search-style task is not.

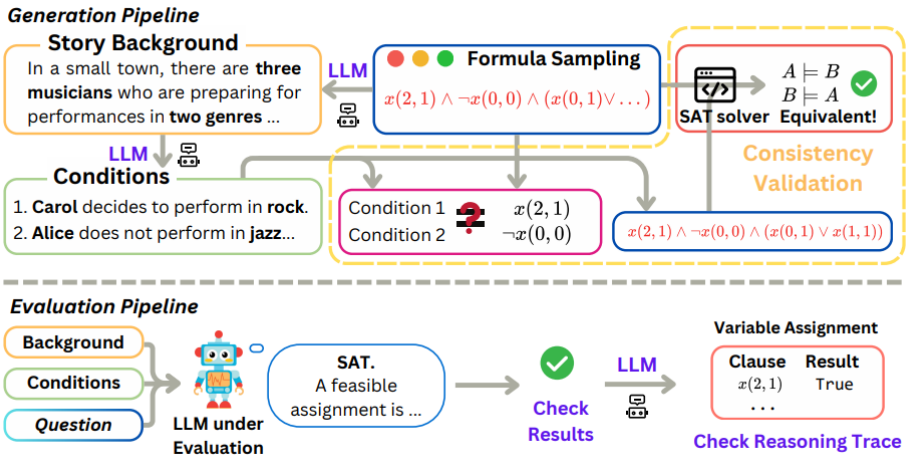

Figure 6.1: Overview of the SATBench methodology. SATBench generates logical puzzles from random SAT formulas.

6.2 ZebraLogic: The Scaling Cliff in Constraint Satisfaction

ZebraLogic [8] attacks an adjacent corner of the logical-reasoning landscape. The benchmark consists of "Einstein-style" or "zebra" puzzles, where a small number of objects must be assigned to a small number of slots subject to a list of constraints ("the doctor lives next to the green house;" "the cat-owner drinks tea;" ...). Such puzzles are easy to generate from a known ground-truth assignment by sampling a subset of true constraints. Difficulty is controlled by the grid size: a 3×3 grid (three persons, three properties) is easy, a 5×5 grid is hard, and beyond that the grids become very hard even for humans.

The headline result of ZebraLogic is a sharp *scaling cliff*. Frontier models solve 3×3 and 4×4 grids respectably, but accuracy collapses on 5×5 and beyond, even with substantial TTS budgets and self-consistency. The collapse is too abrupt to be explained by "the problem just got harder"; it is more consistent with the model running out of patterns it has memorised and failing to generalise. ZebraLogic therefore gives a single, easily reproducible illustration of the broader thesis that current LLMs lack a general constraint-satisfaction capability. A useful diagnostic from the paper is that giving the model intermediate constraints – a "hint" that prunes the search space – recovers most of the lost accuracy, which again suggests the bottleneck is search, not constraint understanding.

6.3 Building Causal Graphs

Tu et al. [9] ask whether LLMs can build causal graphs from natural-language descriptions. The task is to read a passage describing a scientific or medical domain and to output a directed graph whose nodes are the entities mentioned and whose edges encode the causal relationships. This skill is essential for any LLM that wants to operate in scientific or epidemiological settings, where confusing correlation with causation is dangerous.

The findings are not encouraging. LLM-built causal graphs are inconsistent across paraphrases of the same passage; they show strong directional biases (the model prefers the direction of causality that is more frequent in the training data, no matter what the

textual evidence says); and they hallucinate edges that the passage does not support when the entity pair is well-known in the domain. The authors document, for example, that LLMs often invert cause and effect when the inverted relationship is more heavily represented on the web, even when the passage explicitly states the correct direction. The pattern is consistent with the broader story: correlation-style retrieval over the training corpus dominates over genuine inference from the input.

6.4 Agentic Hazards: Document Corruption Under Delegation

The previous works concern reasoning in the narrow sense: the model is asked a question and produces an answer. A growing class of applications, however, places the LLM in an *agentic* role: it is delegated a task that it has to execute on its own over an artefact, typically a document. The work in [10] asks what can go wrong in such delegations.

The findings are concerning even for users who do not care about "reasoning" in the abstract. When LLMs are asked to edit, refactor, summarise, or otherwise transform documents on the user's behalf, they often introduce silent corruptions: facts are dropped, numerical values are rounded, citations are misattributed, and sometimes content is just invented. Crucially, these corruptions are *below the user's threshold of detection*: the document looks fine, reads fluently, and would pass a casual look, but it no longer faithfully represents the input. From a reasoning point of view, this is the agentic version of GSM-NoOp: the model has used information that should have been left alone (or vice versa), and the user has no easy way to tell.

The implication is that the evaluation methodology for LLM agents must include *faithfulness* probes alongside the usual task metrics. A document-editing agent that produces fluent, on-topic output but corrupts the underlying facts is dangerous in proportion to how much the user trusts it; in regulated domains (medical records, legal contracts, financial filings) the cost of silent corruption can be very high.

Chapter 7

Choosing the Right TTS Strategy

Given the long list of TTS strategies and a fixed compute budget, which one should we use? Agarwal et al. [13] answer this question with a large-scale, controlled comparison across eight LLMs and a range of tasks, with all strategies normalised to the same token budget. Their findings boil down to three takeaways.

No single strategy dominates. On any given task there is a best strategy, but the identity of that best strategy changes from task to task. Majority voting works very well on tasks with a small answer space (multiple choice, short numerical answers). Best-of-N with a learned verifier works very well on tasks where the verifier is well-calibrated. Beam search works very well on tasks where the correct answer is reached through one coherent line of deliberation. Recommending one strategy across the board is therefore a bad idea.

Model class matters. "Thinking" models (those tuned to emit very long, self-revising traces, like GPT-o1 and DeepSeek-R1) benefit from *sequential* scaling: extending the trace gives better answers because the model knows how to use the extra room. "Standard" models (those not specifically tuned for long-horizon deliberation) benefit more from *parallel* scaling: sampling several short traces and aggregating them gives a better return on the same token budget. The wrong choice can spend many extra tokens for almost no accuracy gain.

Diminishing returns kick in early. For most strategy/model/task combinations, accuracy as a function of compute budget is concave: a lot of improvement up to a few hundred or a few thousand tokens, then a long flat tail. Past the knee of the curve, more compute helps very little. Beam search, despite being popular, often underperforms simpler strategies (majority voting, best-of-N) at equal compute, because it spends tokens on exploring branches that the verifier cannot rank well.

Table 7.1: TTS strategy selection cheatsheet.

Setting	**Recommended**	**Avoid**
Thinking model, complex math	Long sequential trace, optionally MCTS-guided	Aggressive parallel sampling (wastes the budget)
Standard model, arithmetic / QA	Majority voting / best-of-N with verifier	Pure beam search (poor verifier alignment)
Tight token budget	Best-of-N with N=4–8	Long sequential or large-N parallel

The methodological lesson is that any reasoning evaluation that uses one fixed TTS recipe across all models is potentially misleading. Two models with different architectures

or different fine-tuning may have very different optimal recipes; comparing them under one shared recipe biases the comparison toward the model whose recipe was chosen.

Chapter 8

Case Study: The Pencil-Exchange Experiment

To anchor the survey's lessons in a concrete experiment, we designed and ran our own adversarial probe, which we call the *Pencil-Exchange* task. The task is intentionally simple in structure – track who has how many pencils at each timestep – but it is built so that the difficulty knob, the surface changes, and the question types can all be varied independently. It therefore lets us reproduce, in a single controlled setting, several of the failure modes catalogued by the survey: dynamic generation (defeats contamination), state-tracking (stresses long-horizon reasoning), surface paraphrase (stresses robustness to lexical change), and structural interleaving (stresses temporal book-keeping).

8.1 Game Setup and Conditions

The game is parameterised by the number of persons N and the number of timesteps T. The state of the world is a $T \times N$ non-negative integer matrix M, where $M_{t,i}$ is the number of pencils held by person i at timestep t. The conditions of the game are listed below.

C1 – Matrix structure. The state at every timestep is an integer vector of length N. We require an integer-valued state throughout (no fractional pencils).

C2 – Conservation and validity. (a) The total number of pencils across all persons stays the same for all t, equal to the initial total. (b) In any transfer "person A gives B a quantity n at time t," the quantity n does not exceed A's current holding $M_{t-1,A}$. (c) No holding is ever negative.

C3 – Cluster structure. The N persons are partitioned into contiguous clusters, chosen randomly *before* the simulation begins. Pencils circulate *only* within a cluster; no pencil ever crosses a cluster boundary. The clusters are therefore independent sub-games coupled only by the global conservation law.

C4 – Verbalisation templates. Each transfer is rendered into a natural-language sentence by sampling from a pool of 30 paraphrastic templates. Examples include "A gave B n pencils," "B took from A n pencils," "A transferred n pencils to B," "B took possession of n of A's pencils," "n of the pencils that were with A were transported to B," "A gave a messenger n pencils which the messenger then deposited with B," and so on. The pool of 30 templates makes sure the same transfer never reads the same way twice in a single run.

C5 – Numeric / verbal rendering. The transferred quantity n is, with some probability, written as an English numeral ("five") instead of as a digit ("5"). This forces the model to handle both surfaces and is a deliberate stress test inspired by the surface-

robustness literature.

C6 – Temporal interleaving. After all sentences are generated, they are *interleaved* across clusters and timesteps. The natural ordering would be $\langle t_1c_1, t_1c_2, \ldots, t_2c_1, t_2c_2, \ldots \rangle$ (timestep-major) or $\langle t_1c_1, t_2c_1, \ldots, t_1c_2, t_2c_2, \ldots \rangle$ (cluster-major). We deliberately choose a permutation that is neither, while preserving the temporal semantics within each cluster (timesteps still appear in order *within* a cluster). The model has to reconstruct the cluster structure and the timestep ordering from the interleaved text.

Person × Timestep Pencil-Distribution Matrix
(Columns coloured by clusters)

Timestep ↓ (t)	Person (p)														
	(1) Ram	(2) Shyam	(3) Abhi	(4) Ayaan	(5) Sita	(6) Gita	(7) Riya	(8) Abdul	(9) Ravi	(10) Sai	(11) Tina	(12) Rina	(13) Priya	(14) Mahi	(15) Neha
t = 0	2	3	0	6	4	2	1	12	13	0	7	8	0	9	4
t = 1	1	4	3	1	2	4	14	6	17	6	3	8	3	6	4
t = 2	2	5	2	2	3	2	12	0	16	12	4	8	5	6	2
t = 3	6	5	0	0	6	10	10	6	14	8	4	8	8	3	2
t = 4	1	2	3	5	4	5	8	4	14	4	10	8	10	1	2
t = 5	5	4	1	1	1	11	5	10	8	8	6	8	6	0	7
t = 6	4	7	0	0	7	2	8	0	13	6	8	8	3	6	4
t = 7	6	1	1	3	8	0	9	0	13	5	14	8	6	3	4
t = 8	5	0	2	4	14	0	3	0	13	12	5	10	3	6	4
t = 9	5	0	0	6	17	0	0	0	19	0	11	10	6	3	4
t = 10	2	3	3	3	12	2	3	0	12	7	17	4	3	6	4

Cluster 1 (Ram – Ayaan) Cluster 2 (Sita – Riya) Cluster 3 (Abdul – Sai) Cluster 4 (Tina – Rina) Cluster 5 (Priya – Neha)

Figure 8.1: Person-timestep pencil distribution. Columns are persons $(A_1, \ldots, A_N)$; rows are timesteps. Coloured blocks mark the randomly chosen contiguous clusters within which pencils circulate. Within each cluster, the column sums are conserved across timesteps (C2–C3).

8.2 Execution Pipeline

The Python pipeline takes N and T as inputs and proceeds as follows.

1. **Cluster sampling.** A random partition of $\{1, \ldots, N\}$ into contiguous clusters is drawn. The cluster count and sizes are themselves randomised but constrained so each cluster has at least two persons.

2. **Initial distribution.** A random non-negative integer vector of length N is sampled as the initial pencil distribution at $t = 0$, subject to a fixed total budget.

3. **Transfer simulation.** For each $t = 1, \ldots, T$ and each cluster c, a within-cluster transfer is sampled. The transfer is rejected if it violates C2 and re-sampled. The matrix M is updated.

4. **Verbalisation.** Each accepted transfer is rendered into a natural-language sentence by sampling a template from C4 and a numeric/verbal style from C5.

5. **Interleaving.** The full set of sentences is permuted as in C6, while preserving the within-cluster timestep ordering.

6. **Output.** The pipeline emits (a) the full state matrix M in JSON, used as ground truth for grading, and (b) the interleaved natural-language transcript, which is the only artefact shown to the model.

8.3 Question Types

For each generated instance we issue four families of probe question.

- `person_timestep_lookup`: "How many pencils did ⟨person⟩ have at timestep ⟨t⟩?"
- `person_value_timesteps`: "At which timesteps did ⟨person⟩ have exactly ⟨n⟩ pencils? Return all timesteps in ascending order."
- `population_condition`: "At timestep ⟨t⟩, how many people had at least / at most / more than / fewer than ⟨n⟩ pencils?"
- `duration_condition`: "During how many timesteps ($t \geq 1$) did ⟨person⟩ have at least / at most / more than / fewer than ⟨n⟩ pencils?"

The model is required to reply in strict JSON. For example, the schema for `person_value_timesteps` is:

```
{
  "question_type": "person_value_timesteps",
  "person": "<PERSON_NAME>",
  "pencil_count": <INTEGER>,
  "timesteps": [<INTEGER>, ...]
}
```

A response is graded *correct* if and only if the parsed JSON exactly matches the ground truth derived from M. We do not award partial credit, since we are interested in whether the model can do the book-keeping reliably, not whether it is roughly close.

8.4 Experimental Setup

We evaluated **Gemini 3.1 Pro** via the official API. To rule out context-length and rate-limit confounds, we used the API's *context cache* feature: the entire interleaved transcript for each instance was cached server-side, and only the question itself was issued in the live request. The model therefore had the full transcript in context for every probe, and any failure has to come from reasoning over that context, not from truncation or context-window limits.

We swept the difficulty parameter $N = T \in \{50, 75, 80, 85, 90, 95, 100, 110, 120, 125, 150\}$. At every setting we issued 15 probes per question type and computed accuracy as the fraction of strictly correct JSON responses. The analysis here focuses on `person_timestep_lookup` (the simplest probe: a single table lookup) and `person_value_timesteps` (a slightly harder probe: a column-wise scan that returns a list of indices), since these isolate the model's basic state-tracking ability without involving the population/duration aggregations.

8.5 Results

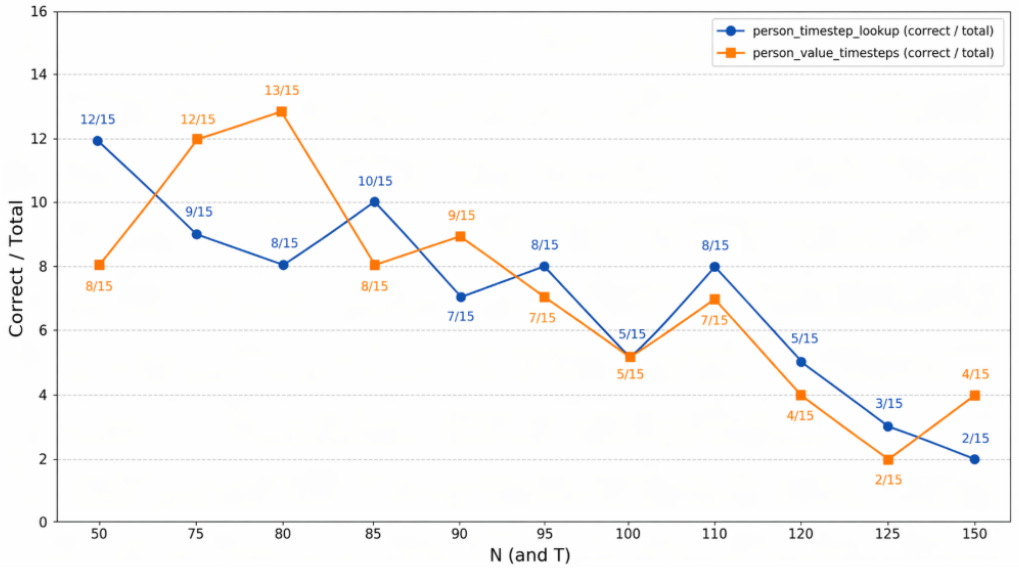

Figure 8.2: Accuracy (correct out of 15) of Gemini 3.1 Pro on the Pencil-Exchange task as a function of the joint scale parameter $N = T$. Blue circles: `person_timestep_lookup`. Orange squares: `person_value_timesteps`.

Table 8.1: Pencil-Exchange accuracy (correct out of 15).

$N = T$	**lookup**	**value_timesteps**
50	12	8
75	9	12
80	8	13
85	10	8
90	7	9
95	8	7
100	5	5
110	8	7
120	5	4
125	3	2
150	2	4

Several patterns come out of Fig. 8.2 and Tab. 8.1.

Average decay with scale. Both question types decay from roughly 12–13/15 at the smallest scale ($N = 50$–80) to roughly 2–4/15 at the largest scale ($N = 125$–150). The overall trend matches what the survey would predict: as soon as problems are made non-trivial, accuracy collapses, even though the underlying task is mechanically simple (running totals on a matrix that fits comfortably in the model's context window).

High variance, non-monotone accuracy. The accuracy curves are strikingly volatile. The lookup curve, for example, goes $12 \to 9 \to 8 \to 10 \to 7 \to 8 \to 5 \to 8$ as $N = T$

moves from 50 to 110. A real algorithmic procedure should produce a smooth, monotonically decreasing accuracy as the problem grows. A zig-zag of this size is the fingerprint of pattern matching: some surface configurations of the transcript happen to land in parts of the model's distribution where a useful template applies, while neighbouring configurations do not. This is exactly the behaviour predicted by the high-variance findings of GSM-Symbolic [6].

Convergence to near-random. Both question types converge to roughly $5/15 \approx 33\%$ around $N = 100$. Given the combinatorial space of possible answers, this is close to what we would expect from a model that has effectively given up on careful book-keeping and is producing locally plausible numeric guesses. From this point on, more compute does not help: the model has hit the ZebraLogic-style scaling cliff [8].

No question-type dominates. Neither lookup nor value-timesteps is uniformly easier. Lookup wins at $N = 50, 85, 95, 110, 120, 125$; value-timesteps wins at $N = 75, 80, 90, 150$. The two probes trade leads in a way that further supports the pattern-matching reading: the model's success depends on which surface form the question happens to take, not on which underlying operation (single lookup vs. column scan) is computationally simpler.

The cache rules out trivial confounds. Because the entire transcript was placed in the model's context cache, the model had literal, byte-for-byte access to all of the information needed to answer every question. Failure cannot be attributed to truncation, paging, or context-window pressure. What the model *lacks* is a reliable mechanism for reading off the cluster structure (C3), undoing the temporal interleaving (C6), and applying the conservation update at each step (C2). This is exactly the structural-reasoning capability that the survey papers find missing in frontier LLMs.

8.6 What the Experiment Confirms

The Pencil-Exchange task reproduces, in a single controlled experiment, three of the central failure modes catalogued by the survey:

1. **Surface fragility** (mirroring GSM-Symbolic [6]): the lexical randomisation in C4–C5 and the structural shuffling in C6 both contribute to high-variance, non-monotone accuracy curves.

2. **Scaling cliff** (mirroring ZebraLogic [8] and SATBench [7]): accuracy collapses from above 80% at small scale to near-random at scales that are still well within the model's context window.

3. **Process–outcome gap** (mirroring DAG-Math [5]): wherever the model gets an answer right, manual inspection shows that the reasoning trace is often inconsistent with the ground-truth update sequence – the model has matched a plausible answer rather than executed the required update.

In short, the experiment gives an in-house confirmation of the survey's main claim: TTS-enhanced frontier LLMs are not, in any structural sense, reasoning their way through this task; they are pattern-matching, and the illusion of reasoning breaks the moment the surface is shaken hard enough.

Chapter 9

Conclusion and Open Challenges

The ten works surveyed here, taken together with our Pencil-Exchange experiment, draw a coherent and somewhat unflattering picture of LLM reasoning circa 2026. Static, answer-only benchmarks substantially over-estimate reasoning ability, and the over-estimation is not the same across models, which distorts leaderboards [4]. The reasoning trace itself, when looked at closely, often contains errors that cancel into a correct final answer [5], and the model's apparent high-level abstraction is largely the recall of memorised code or text templates [12]. Surface perturbations cause large, sometimes catastrophic, drops in accuracy [6]. Even basic arithmetic [11] is unreliable beyond a few digits. Logical reasoning [7, 8] and causal reasoning [9] both show sharp scaling cliffs and systematic biases. Agentic delegation introduces a new class of silent corruption errors [10]. And the choice of TTS strategy itself is non-trivial: no single recipe dominates [13].

TTS-enhanced models are still useful. A model whose final answer is correct ninety percent of the time is valuable even if its reasoning trace is unreliable, as long as the user can verify the answer cheaply. The danger is in domains – scientific, medical, legal, agentic – where the answer is hard to verify and the cost of a mistake is high. In those domains, the absence of trace-aware, dynamic, multi-domain evaluation is not just a methodological inconvenience; it is a safety risk.

We close with four open challenges that come out of the survey and the experiment.

1. **Standardised trace evaluation.** The community needs a shared protocol for measuring trace-level properties – factuality, validity, coherence, and utility – that is both rigorous and cheap to run. PRR from DAG-Math is a good starting point, but it currently applies only to mathematical traces.

2. **Training for robustness, not just for accuracy.** Current fine-tuning rewards correct answers. New objectives should reward correct *reasoning*, possibly using process-reward models trained against DYVAL- or DAG-Math-style references.

3. **Adaptive TTS selection.** Given the strong model-class dependence of optimal TTS recipes, we need principled methods that pick the right strategy at run time without trying everything.

4. **Beyond mathematics.** Process-level evaluation has to be extended to common-sense, causal, multi-modal, and agentic reasoning. The Pencil-Exchange task is one small step in this direction; many more are needed.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[3] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[4] K. Zhu, J. Chen, J. Wang, N. Z. Gong, D. Yang, and X. Xie, "DyVal: Dynamic evaluation of large language models for reasoning tasks," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17167.

[5] Y. Zhang, I. Kuzborskij, J. D. Lee, C. Leng, and F. Liu, "DAG-Math: Graph-of-thought guided mathematical reasoning in LLMs," *arXiv preprint arXiv:2510.19842*, 2025.

[6] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.05229.

[7] T. Wang *et al.*, "SATBench: Benchmarking LLMs' logical reasoning via automated puzzle generation from SAT formulas," *arXiv preprint arXiv:2505.14615*, 2025.

[8] B. Y. Lin *et al.*, "ZebraLogic: On the scaling limits of LLMs for logical reasoning," *arXiv preprint arXiv:2502.01100*, 2025.

[9] S. Tu, K. Aberer, and B. Schölkopf, "Can large language models build causal graphs?," *arXiv preprint arXiv:2303.05279*, 2023.

[10] "LLMs corrupt your documents when you delegate." arXiv preprint arXiv:2604.15597, 2024.

[11] Z. Yuan, H. Yuan, C. Tan, W. Wang, and S. Huang, "How well do large language models perform in arithmetic tasks?," *arXiv preprint arXiv:2304.02015*, 2023.

[12] M. Zhou, F. Bardoliya, V. Gupta, and D. Roth, "Program-of-thought reveals LLM abstraction ceilings," in *Findings of the Association for Computational Linguistics: EACL*, pp. 4911–4919, 2026.

[13] A. Agarwal, A. Sengupta, and T. Chakraborty, "The art of scaling test-time compute for large language models," *arXiv preprint arXiv:2512.02008*, 2025.